

EventBridge

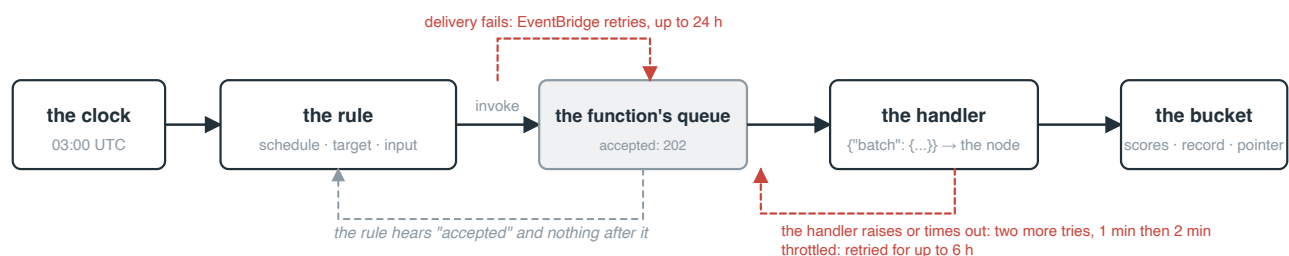
A one-page primer: a clock, a target and an input, and what we use it for

Written for the *score every account* lab. The platform in the course is AWS; the mechanism is the same on every cloud. Names are October 2026 and will change.

What it is

A **rule** is a schedule, a target, and the input the target receives. The clock reaches the minute, EventBridge hands the input to the target and is told "accepted", and that is the end of its involvement. It does not wait, does not learn what happened, and keeps no record. In the lab:

schedule	<code>cron(0 3 * * ? *)</code> : 03:00 UTC nightly. Six fields, UTC, <code>?</code> in the day you are not using; or <code>rate(5 minutes)</code> . Nothing faster than once a minute.
target	the scoring function, invoked asynchronously
input	<code>{"batch": {"key": "accounts/nightly.jsonl"}}</code> : the handler sees this, not an HTTP request



so the node is written to be run twice

every row carries the run id; the scores file and the record for that id are replaced, never appended;
the pointer moves only when the check passes. A retry, a duplicate delivery, or a hand re-run all land in the same place.
When every retry is exhausted the event is dropped, and the morning reader finds the pointer still on yesterday.

What we use it for

- **The nightly run.** Score every account, rebuild the feature table, re-index the corpus, retrain. One rule per job; the job is a function or a state machine.
- **The scheduled check.** Run the eval harness against the pinned model at the same hour every day: the vendor drift canary. The rule is the only part of it that is not your code.
- **Reacting to a file landing.** A rule can match an event instead of the clock: an object written to a bucket starts the job that reads it. Same object, a pattern in place of a schedule.
- **Watching it fire.** `rate(1 minute)` and the run records in the bucket show the mechanism in the time it takes to read this page.

What it does to your node

Delivery is retried for up to 24 hours if the invoke call fails; the function's own queue retries a handler that raises twice more, one minute then two minutes later; and the queue can deliver the same event twice with no error anywhere. So the node must be safe to run twice for the same run id: replace the night's file and record, never append, and move the pointer only when the check passes. When every retry is exhausted the event is dropped and nothing tells you; the morning's reader finds yesterday's pointer.

What it is not

It is not an orchestrator. It has no steps, no order between its targets, no retry of *your* steps, and no run record. Everything on the lecture's orchestrator slide that is not "start the job on the schedule" is what the lab's node writes by hand and what Airflow, Dagster, Prefect or Step Functions would write for every step. The rule costs nothing to fire; the function's running time is the bill.